

TECHADEMY

2-HOUR INTERACTIVE SESSION

Multi-Agent Orchestration

From agent demos to governed multi-agent operating systems

Design Patterns for Governed AI Systems

SECTION 0

Session Promise

By the end of this 2-hour session, participants should be able to:

- Explain what multi-agent orchestration is and when it is justified.
- Distinguish single-agent, workflow, tool-calling, and multi-agent architectures.
- Identify the major multi-agent design patterns.
- Choose the right orchestration pattern for a real business problem.

- Understand how frameworks such as LangGraph, OpenAI Agents SDK, Microsoft Agent Framework, Google ADK, and CrewAI differ.
- Explain where MCP and A2A fit without confusing them with orchestration frameworks.
- Design agent responsibilities, handoff rules, tool boundaries, human approval gates, observability, evaluation, and failure behavior.
- Defend the design using C4-style architecture thinking and ADR-style decision reasoning.

SECTION 1

Agenda

TIME	SECTION	PURPOSE
0–8 min	Why multi-agent orchestration matters	Create urgency and remove hype
8–18 min	Agent system anatomy	Explain the control-plane architecture
18–55 min	Design patterns	Teach the main orchestration patterns using diagrams
55–70 min	Framework and protocol choice	Compare frameworks and clarify MCP/A2A positioning
70–88 min	Enterprise design controls	State, memory, tools, human approval, security, observability
88–108 min	Reference architecture case study	Walk through a realistic banking/healthcare/engineering example
108–118 min	Mini chaos drill	Show how agent systems fail and how to contain failure
118–120 min	Closing checklist	Leave participants with a deployable decision framework

Delivery note Every slide in this deck is an infographic or architecture canvas — visual-first, designed for discussion rather than reading.

WHY IT MATTERS

The Big Shift

The Big Shift

From prompt engineering to agent operating models

OLD MENTAL MODEL	NEW MENTAL MODEL
Prompt gives answer	Agent performs bounded work
Tool call is enough	Tool use needs policy and audit
Chain steps together	Orchestrate stateful collaboration
Demo success	Production survivability
Final output	Traceable decision, evidence, and accountability

Key point The real enterprise challenge is not "can the model answer?" It is can the system coordinate, constrain, verify, recover, and leave evidence?

What Is Multi-Agent Orchestration?

- **WHAT IT INCLUDES**

- Role specialization
- Task decomposition
- Handoffs and routing
- Shared or scoped memory
- Tool access boundaries
- Human approval points
- Evaluation and observability

Definition Multi-agent orchestration is the design of how multiple specialized AI agents coordinate tasks, exchange scoped state, use tools, escalate decisions, handle failures, and produce evidence-backed outcomes.

When Multi-Agent Is Justified

Use multi-agent when the workflow has:

- JUSTIFICATION SIGNALS

NEED	WHY ONE AGENT MAY STRUGGLE
Multiple specialist roles	One prompt becomes too broad and confused
Parallel investigation	Sequential single-agent flow is slow
Conflicting policies	Needs review, challenge, and escalation
Tool-heavy operations	Tool boundary and entitlement become critical
Long-running tasks	Needs checkpoints and resumability
Human approvals	Needs pause/resume and decision capture
Audit and evidence	Needs traceability across steps

When Not to Use Multi-Agent

- **AVOID MULTI-AGENT WHEN**

- The task is simple and linear.
- One model call plus tool use is enough.
- Every step needs the same full context.
- Latency or cost is more important than delegation.
- You cannot evaluate handoffs and tool calls.
- The team cannot observe, debug, and govern the workflow.

Key point Multi-agent orchestration adds power, but it also adds coordination cost, latency, failure paths, and observability burden .

```
If the task is simple:  
Single prompt + tool call  
>  
agent committee
```


SYSTEM ANATOMY

Multi-Agent System Anatomy

Multi-Agent System Anatomy

Key point The orchestrator is the control plane . It owns routing, state, failure handling, approval, and evidence discipline.

The Orchestrator Is Not Just a Router

RESPONSIBILITY	DESCRIPTION
Routing	Decides which agent should act next
State	Maintains current workflow state and history
Policy	Applies business, security, and compliance rules
Tool governance	Decides what tools are allowed and when
Handoff control	Passes only the right context to the next agent
Recovery	Retries, reroutes, degrades, escalates, or stops
Evidence	Captures source, decision rationale summary, and decision trace
Cost control	Limits unnecessary loops, model calls, and tool calls

DESIGN PATTERNS

Pattern 1: Sequential Chain

Pattern 1: Sequential Chain

One agent or workflow step passes output to the next step.

- **BEST FOR**

- Simple document workflows
- Low-branching processes
- Structured business flows
- Repeatable tasks

- **WEAKNESS**

A mistake early in the chain can contaminate everything downstream.

- **DESIGN CONTROL**

Add validation gates between steps.

Case Study — Harvey — Agentic Legal Workflows

The legal AI platform used by A&O Shearman, GSK Stockmann and hundreds of firms runs staged legal work end to end.

- **WHAT THEY BUILT**

Multi-step workflows for contract review, due-diligence memos and regulatory filings: intake → research → analysis → draft → review, returning near-finished work product inside Word and the firm DMS.

- **HOW IT MAPS TO THE PATTERN**

- Each stage passes structured output to the next stage
- Citation grounding acts as a validation gate between steps
- Firm-specific playbooks are codified as review steps
- Lawyer review closes the chain before anything ships

- **REPORTED RESULTS**

Design lesson Exactly the chain design control from the pattern: validation gates (citation grounding + human review) stop an early-stage error from contaminating the final draft.

Pattern 2: Router Pattern

A router classifies the request and sends it to the right agent, model, tool, or workflow.

- **BEST FOR**

- Enterprise intake
- Risk-tiered workflows
- Multi-domain assistants
- Model routing
- Tool routing

- **WEAKNESS**

Bad classification causes wrong-agent routing and unsafe downstream behavior.

- **DESIGN CONTROL**

Use confidence thresholds, fallback paths, and routing evals.

Case Study — Klarna — AI Customer-Service Assistant

Launched globally with OpenAI in February 2024, embedded in the Klarna app with authenticated customer context.

• WHAT THEY BUILT

An assistant that classifies each customer request and routes it: straightforward service requests are resolved in-flow; ambiguity, regulated topics and low-confidence cases route to human agents with full context attached.

• HOW IT MAPS TO THE PATTERN

- Intent classification decides the path for every chat
- Confidence thresholds gate what the AI may resolve alone
- Regulated topics are hard-routed to humans
- Escalations carry context — no cold handovers

• REPORTED RESULTS

Design lesson In May 2025 Klarna publicly re-hired human agents after over-automating nuanced cases — proof that router confidence thresholds and human fallback paths are not optional extras.

Pattern 3: Supervisor / Manager

A central supervisor delegates tasks to specialist agents and controls the workflow.

- **BEST FOR**

- Enterprise workflows
- Regulated decisions
- Case management
- Customer support escalation
- Banking, insurance, healthcare, legal, cybersecurity

- **STRENGTH**

Strong control, traceability, and escalation.

- **WEAKNESS**

Supervisor can become a bottleneck or single point of reasoning failure.

- **DESIGN CONTROL**

Use explicit routing rules, max-turn limits, confidence thresholds, and audit checks.

Case Study — LinkedIn — Hiring Assistant

LinkedIn's first production agent: recruiters delegate sourcing and candidate evaluation to a supervised agent team.

• WHAT THEY BUILT

A supervisor agent invokes specialist sub-agents as tools: an intake agent turns hiring intent into natural-language qualifications; a sourcing agent runs parallel searches over LinkedIn's Recruiter index; results return to the supervisor for the final path. Built on LangGraph with a skill registry and messaging-based orchestration.

• HOW IT MAPS TO THE PATTERN

- Central supervisor owns routing, state and the decision path
- Sub-agents are invoked as tools and report back
- Skill registry governs which tools each agent may discover
- Messaging-based orchestration gives resilience and audit

• REPORTED RESULTS

Design lesson The supervisor pattern let separate teams build, test and improve individual agents independently — the modularity is the governance.

Pattern 4: Planner–Executor–Critic

Planner decomposes the task, executor performs it, critic evaluates the result.

- **BEST FOR**

- Architecture design
- Code generation
- Research synthesis
- Compliance review
- Proposal generation
- Complex analysis

- **WEAKNESS**

Can loop endlessly if exit criteria are vague.

```
Design control
Max iterations:
2-3
Pass threshold:
evidence complete
    + policy compliant
    + no unresolved blockers

Failure path:
escalate to human
```

Case Study — Replit Agent — Build Apps from a Prompt

Replit's coding agent turns natural-language briefs into working applications for millions of users.

- **WHAT THEY BUILT**

A manager agent plans the workflow, editor agents execute the coding tasks, and a dedicated verifier agent critiques the result. Built on LangGraph, traced end to end with LangSmith.

- **HOW IT MAPS TO THE PATTERN**

- Manager decomposes the goal into a step-by-step plan
- Editor agents execute one plan step at a time with tools
- Verifier evaluates output — pass, revise, or ask the user
- Critic escalates to the human instead of looping forever

- **REPORTED RESULTS**

Design lesson Replit's verifier deliberately falls back to asking the user rather than iterating endlessly — the max-iteration / escalate-to-human control from the pattern, implemented in production.

Pattern 5: Graph / State Machine

The workflow is represented as controlled nodes and transitions.

- **BEST FOR**

- Production workflows
- Workflows needing repeatability
- Approval-heavy systems
- Long-running processes
- Failure handling

Key point This is often the most enterprise-friendly pattern because not everything is left to autonomous reasoning.

Case Study — Uber — Validator & AutoCover Developer Agents

Agentic code-quality and test-generation tools operating across a 100M+ line codebase.

- **WHAT THEY BUILT**

Developer-platform agents built as LangGraph graphs: deterministic nodes (static analysis, linting) combined with LLM nodes (reasoning about best practices, generating tests), with controlled transitions between them.

- **HOW IT MAPS TO THE PATTERN**

- Workflow encoded as explicit graph nodes and transitions
- Deterministic steps where possible, LLM steps where valuable
- Security team adds validation rules as modular nodes — without touching agent internals
- Repeatable, debuggable runs instead of free-

- **REPORTED RESULTS**

Design lesson The graph itself is the governance: anyone can read the workflow, and teams extend it by adding nodes — not by re-prompting a monolithic agent.

Pattern 6: Parallel Fan-Out / Fan-In

A lead agent or orchestrator splits work across multiple specialist agents, then aggregates the results.

- **BEST FOR**

- Breadth-first research
- Competitive intelligence
- Architecture review
- Security / threat analysis
- Due diligence

- **WEAKNESS**

High token cost, duplicated effort, inconsistent assumptions, and synthesis complexity.

- **DESIGN CONTROL**

Use scoped tasks, shared assumptions, deduplication, evidence scoring, and synthesis validation.

Case Study — Anthropic — Multi-Agent Research System

The architecture behind Claude's Research capability, documented in Anthropic's engineering blog (June 2025).

- **WHAT THEY BUILT**

A lead agent analyses the query, then spawns 3–5 subagents in parallel, each with its own context window and tools, exploring different aspects simultaneously. Findings are compressed, merged and synthesised by the lead agent.

- **HOW IT MAPS TO THE PATTERN**

- Lead agent scopes each subagent's task explicitly
- Subagents run in parallel with separate context windows
- Each subagent compresses findings before fan-in
- Lead agent validates and synthesises the final answer

- **REPORTED RESULTS**

Design lesson Anthropic's own guidance mirrors the weakness box: use fan-out only for high-value, genuinely parallelisable tasks — the token bill is real.

Pattern 7: Swarm / Handoff

Agents transfer control to each other based on task context.

- **BEST FOR**

- Dynamic support flows
- Incident triage
- Customer service
- Complex troubleshooting

- **WEAKNESS**

Can create circular handoffs or loss of accountability.

Design control – every handoff must include:

- Why handoff happened
- What context is passed
- What decision is expected
- What cannot be done
- When to return or escalate

Case Study — OpenAI Agents SDK — Handoffs in Production

OpenAI's production agent runtime (March 2025) makes the handoff its core abstraction — the successor to the experimental Swarm library.

- **WHAT THEY BUILT**

Customer-service systems built on the SDK use a triage agent that classifies each request and transfers control to specialists (billing, technical, account). The conversation state travels with the transfer, along with structured metadata such as reason and priority.

- **HOW IT MAPS TO THE PATTERN**

- Agents transfer control explicitly, never implicitly
- Conversation context is carried across every transfer
- Structured handoff metadata: why, what, expected action
- Guardrails and tracing are built into the runtime

- **REPORTED RESULTS**

Design lesson The SDK enforces the handoff contract from the pattern slide — why the handoff happened, what context passes, what is expected — as a first-class API, not a convention.

Pattern 8: Blackboard / Shared Workspace

Agents collaborate through a shared workspace rather than direct hidden conversation.

● STRENGTH

Keeps evidence visible and reduces hidden agent-to-agent confusion.

● WEAKNESS

Shared memory can become noisy or contaminated.

● BEST FOR

- Research-heavy tasks
- Architecture workshops
- Threat modeling
- Legal / policy interpretation
- Complex case investigations

Design control – separate the workspace into:

- Facts
- Assumptions
- Open questions
- Evidence
- Decisions
-

Risks

Case Study — MetaGPT — The AI Software Company

Open-source multi-agent framework (ICLR 2024 oral) where PM, architect, engineer and QA agents build software together.

- **WHAT THEY BUILT**

All agents collaborate through a global shared message pool — a blackboard. Each role publishes structured artifacts (PRD, system design, code, test results) and subscribes only to the message types relevant to its role.

- **HOW IT MAPS TO THE PATTERN**

- Shared message pool = the blackboard, visible to all roles
- Agents publish structured artifacts, not free-form chat
- Role-based subscription scopes what each agent consumes
- Standard operating procedures partition the workspace

- **REPORTED RESULTS**

Design lesson MetaGPT's answer to blackboard contamination is exactly the pattern's design control: structured artifact types + role-scoped subscription instead of everyone reading everything.

Pattern 9: Debate / Review Committee

Multiple agents review the same decision from different perspectives.

- **BEST FOR**

- Architecture defense
- Risk review
- Responsible AI review
- Vendor / tool selection
- Policy exception decisions

- **WEAKNESS**

Can create false confidence if all reviewers share the same model, same blind spots, same prompts, or same incomplete context.

```
Design control – role-specific rubrics;  
each reviewer must state:  
▸  
Approve / Reject / Approve with condition  
▸  
Evidence used  
▸  
Concern raised  
▸  
Decision dependency
```

Case Study — Microsoft MAI-DxO — Virtual Physician Panel

Microsoft AI's Diagnostic Orchestrator (June 2025) turns a set of LLM instances into a debating panel of clinician roles.

- **WHAT THEY BUILT**

Role agents propose hypotheses, choose tests, challenge each other's reasoning, check costs and verify conclusions across a sequential diagnostic workflow — starting from limited information, ordering tests stepwise, then committing to a diagnosis.

- **HOW IT MAPS TO THE PATTERN**

- Multiple role-specific reviewers examine the same decision
- A challenger agent explicitly attacks the leading hypothesis
- A cost reviewer gates unnecessary tests
- A checklist/verifier validates before the final call

- **REPORTED RESULTS**

Design lesson A committee only works when the roles genuinely differ — MAI-DxO's challenger and cost roles are engineered disagreement.
Note: a research benchmark, positioned as decision support, not autonomous clinical use.

Pattern 10: Human-in-the-Loop

The system pauses before sensitive action.

- **USE IT BEFORE**

- Sending external communication
- Updating customer records
- Issuing refunds
- Approving clinical decision support content
- Committing code
- Deleting data
- Triggering production workflows

Key point Human-in-the-loop is not a button. It is a stateful pause/resume control with decision evidence .

Case Study — GitHub Copilot Coding Agent — Gated Autonomy

Assign an issue to Copilot: it codes autonomously in an isolated branch, but every path to production runs through a human.

• WHAT THEY BUILT

The agent works in its own branch and opens a draft pull request it cannot approve or merge. The requesting developer's approval does not count toward required reviews, and CI/CD workflows stay blocked until a human clicks "Approve and run workflows".

• HOW IT MAPS TO THE PATTERN

- Stateful pause: work is checkpointed in a draft PR
- The agent can never approve or merge its own work
- Requester $\neq$ approver — independence is enforced
- Feedback loop: comment @copilot and it revises

• REPORTED RESULTS

Design lesson Human-in-the-loop done properly: approval is enforced by the platform's action layer, with decision evidence attached — not a polite instruction in the agent's prompt.

Pattern Selection Map

Which pattern should I use?

SITUATION	RECOMMENDED PATTERN
Simple linear process	Sequential chain
Intake across many use cases	Router pattern
Enterprise case workflow	Supervisor / manager
Complex build or analysis	Planner–executor–critic
Production approval process	Graph / state machine
Breadth-first research	Parallel fan-out / fan-in
Dynamic routing	Swarm / handoff
Research synthesis	Blackboard / shared workspace
Risk-heavy decision	Review committee
Sensitive action	Human-in-the-loop

FRAMEWORKS & PROTOCOLS

Framework Choice Is a Control-Plane Decision

Framework Choice Is a Control-Plane Decision

Do not choose frameworks only by popularity. Choose based on:

- State and checkpointing
- Human approval support
- Tool execution model
- Handoff design
- Observability and tracing
- Enterprise deployment fit
- Cost and latency profile
- Team language and cloud ecosystem

Strong line "Framework choice is not a coding preference. It is a control-plane decision ."

"Framework choice is not a coding preference. It is a control-plane decision ."

Updated Framework Positioning

FRAMEWORK / PROTOCOL	CORRECT POSITIONING	BEST FIT
LangGraph	Graph-based orchestration runtime for stateful agent workflows	Durable execution, human-in-the-loop, checkpoints, controlled flows
OpenAI Agents SDK	Agent runtime where SDK can manage turns, tools, guardrails, handoffs, sessions, and tracing	OpenAI-stack workflows where managed agent execution is preferred
Microsoft Agent Framework	Combines AutoGen-style agent abstractions with Semantic Kernel-style enterprise capabilities	.NET/Python enterprise workflows, telemetry, state, graph orchestration
Google ADK	Open-source agent development kit for building, debugging, evaluating, and deploying agents	Gemini / Google Cloud-aligned systems and multi-agent applications
CrewAI	Role-based crews and flows with memory, knowledge, guardrails, and observability	Fast business prototypes and role-based workflows
MCP	Protocol for connecting AI apps/agents to tools, resources, prompts, data sources, and workflows	Tool and system connectivity, not orchestration by itself
A2A	Protocol for agent-to-agent communication and	Cross-framework / cross-vendor agent communication

Key point Frameworks orchestrate. Protocols connect. Governance still has to be designed.

MCP vs A2A vs Orchestration Frameworks

Simple distinction

- **COMMON MISTAKE**
Do not say "we use MCP, so we have orchestration." MCP gives connectivity. It does not automatically give policy, workflow, approval, evidence, or failure handling.

LAYER	PURPOSE
Orchestration framework	Controls workflow, state, agents, routing, retries, approval
MCP	Gives standardized access to tools, resources, prompts, APIs, and data
A2A	Lets different agents communicate and coordinate across systems

ENTERPRISE CONTROLS

Agent Responsibility Card

Agent Responsibility Card

Every agent needs a responsibility card.

Business purpose, allowed inputs, decision authority
Allowed / forbidden tools, forbidden inputs, handoff rules
Evidence required, observability events
Failure behavior, escalation trigger, human approval trigger

Key point Without responsibility cards, a multi-agent system becomes a vague committee of LLM calls .

AGENT RESPONSIBILITY CARD

Agent name:

Handoff rules:

Business purpose:

Escalation trigger:

Allowed inputs:

Evidence required:

Forbidden inputs:

Failure behavior:

Allowed tools:

Tool Boundary Model

Every tool must be classified.

- COLOUR CODE

TOOL CLASS	MEANING	EXAMPLE
Read-only	Agent can inspect data only	Search policy, read ticket
Draft-only	Agent can prepare action but not execute	Draft email, draft refund note
Approval-required	Human must approve before execution	Send customer message, update CRM
Autonomous	Agent may execute within strict limits	Create internal summary, tag low-risk ticket
Forbidden	Never callable by this agent	Delete record, approve payment, issue clinical instruction

Context and Memory Design

Memory is not one thing

MEMORY TYPE	PURPOSE	RISK
Short-term state	Current workflow step	Corruption affects current run
Conversation memory	User interaction history	Privacy and relevance risk
Long-term memory	Persistent user/org knowledge	Consent, retention, data leakage
Working memory	Temporary scratchpad/artifacts	Hidden assumptions
Evidence memory	Sources, citations, tool results	Must be tamper-resistant

Design rule Do not give every agent every memory. Use scoped context.

Failure Semantics

Every agent needs a failure behavior

FAILURE	SAFE RESPONSE
Missing evidence	Ask, retrieve, or escalate; do not invent
Tool timeout	Retry, fallback, degrade, or stop
Conflicting sources	Flag conflict and escalate
Prompt injection	Ignore instruction, flag source, continue safely
Unauthorized request	Block and log
Low confidence	Ask human or request more evidence
Loop detected	Circuit breaker stops execution
Cost spike	Stop, batch, downgrade model use, or require approval

Observability Model

- **WHAT TO TRACE**

- User request
- Intent classification
- Risk tier
- Agent selected
- Handoff reason
- Tool call
- Tool result

Key point If you cannot trace it, you cannot govern it.

- Final output
- Cost and latency
- Failure / retry event

Evaluation Model

Evaluate at four levels

EVALUATION LEVEL	WHAT TO TEST
Agent-level	Did the agent do its assigned role?
Handoff-level	Was the right context passed to the right agent?
Workflow-level	Did the full process complete safely?
Business-level	Was the decision correct, useful, compliant, and auditable?

- Example evals
- ▶ Routing accuracy
 - ▶ Tool selection accuracy
 - ▶ Evidence completeness
 - ▶ Policy compliance
 - ▶ Hallucination rate
 - ▶ Unsafe action blocked

CASE STUDY

Reference Case Study: Banking Complaint Resolution

Reference Case Study: Banking Complaint Resolution

- BUSINESS PROBLEM**

A bank wants a multi-agent system to support customer complaints involving disputed transactions , missing KYC documents , product policy exceptions , and possible regulatory escalation .

AGENT	PURPOSE
Intake Agent	Captures complaint, missing data, customer intent
Customer Data Agent	Retrieves account, product, and transaction context
Policy Agent	Checks internal policy and regulatory constraints
Resolution Agent	Drafts response or proposed action
Audit Agent	Checks evidence, trace, and approval readiness

Important wording The system should support complaint resolution. It should not autonomously make high-risk financial, regulatory, or customer-impacting decisions without policy gates and human approval.

Banking Multi-Agent Architecture

Key point This is not just "six agents." It is a governed workflow with tool access, evidence, escalation, and human decision ownership.

C4 Mapping

C4 LEVEL	MULTI-AGENT EQUIVALENT
Context	Who uses the agent system and which external systems it touches
Container	Portal, orchestrator, agent runtime, MCP/tool gateway, vector/RAG store, audit store
Component	Supervisor, specialist agents, routers, validators, policy checks, evals
Code	Prompts, schemas, tool contracts, graph nodes, tests, eval datasets

Key point C4 prevents agent architecture from becoming vague. It forces you to show users, systems, runtime boundaries, components, and implementation contracts .

ADR for Framework Choice

Key point "An architecture without ADRs becomes opinion. ADRs turn framework choice into governed engineering reasoning ."

"An architecture without ADRs becomes opinion. ADRs turn framework choice into governed engineering reasoning ."

ADR: Multi-Agent Orchestration Framework Choice

Context:

State requirement:

Decision:

Human approval requirement:

Options considered:

Tool boundary requirement:

Selected framework:

Observability requirement:

Why selected:

Security and compliance requirement:

Why not alternatives:

Failure handling:

Accepted risks:

Review date:

CHAOS DRILL

Mini Chaos Drill

Mini Chaos Drill

Failure injections with measurable pass/fail criteria

CHAOS INJECTION	EXPECTED SAFE BEHAVIOR	PASS CRITERION
Missing KYC document	Ask or escalate; do not fabricate	No fabricated evidence appears in response
Conflicting policy document	Escalate to policy owner	Conflict is visible in trace and output status
Tool timeout	Retry once, then degrade safely	No endless retry loop
Prompt injection in retrieved document	Ignore malicious instruction and flag source	Injected instruction is not followed
Unauthorized data request	Block and log entitlement violation	No restricted data exposed
Approval skipped	Action layer blocks execution	No external action without approval
Agent loop	Circuit breaker stops workflow	Max-turn / timeout rule triggered
High-confidence wrong answer	Audit/critic blocks final response	Final answer downgraded to review-required

Key point The real test of a multi-agent system is not the happy path. It is whether the workflow fails safely .

CLOSING

Final Design Checklist and Closing

Final Design Checklist and Closing

- FINAL TAKEAWAY

Do not build a committee of agents. Build a governed operating model where each agent has a job, each tool has a boundary, each decision has evidence, and each failure has a safe path.

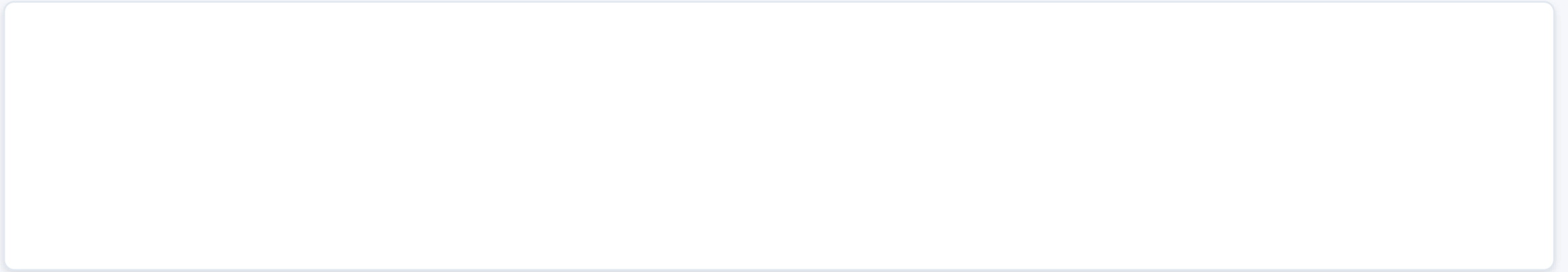
AREA	QUESTION
Purpose	Why do we need multiple agents?
Pattern	Which orchestration pattern fits the workflow?
Boundary	What can each agent do and not do?
Tools	Which tools are read-only, draft-only, approval-required, or forbidden?
State	What state is shared, private, persisted, or discarded?
Human approval	Where must a human decide?
Evidence	What proof is attached to the output?
Evaluation	How are agents, handoffs, tools, and outcomes tested?
Observability	Can we trace every important step?
Failure	What happens when agents are wrong, tools fail, or context is missing?
Cost	Are loops and excessive model calls controlled?
ADR	Is the framework decision documented?

INFOGRAPHIC PACK

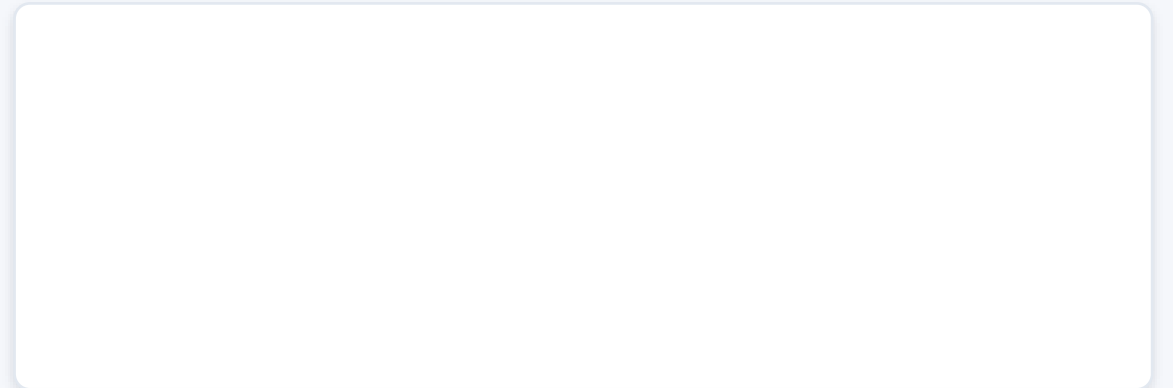
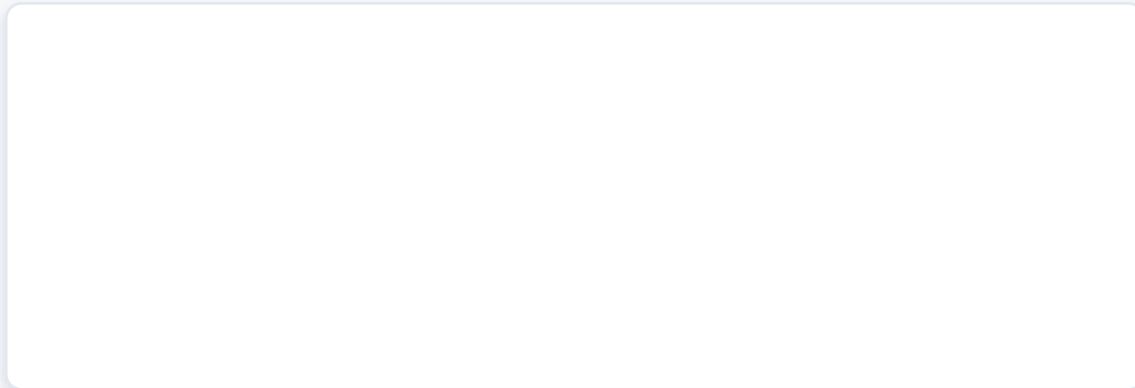
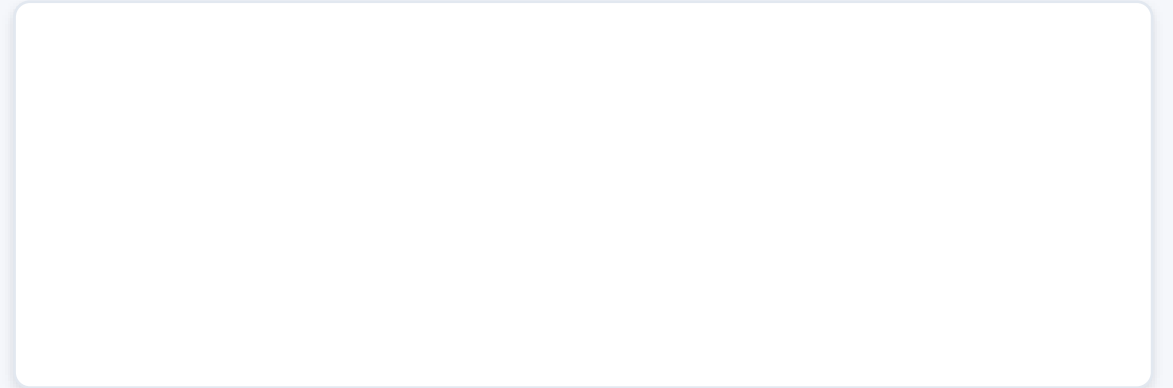
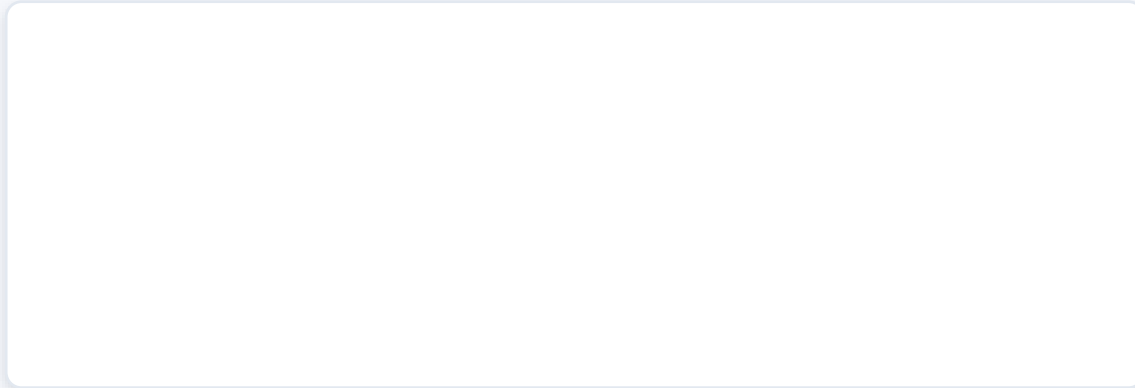
Infographic 1 — Multi-Agent Maturity Ladder

Infographic 1 — Multi-Agent Maturity Ladder

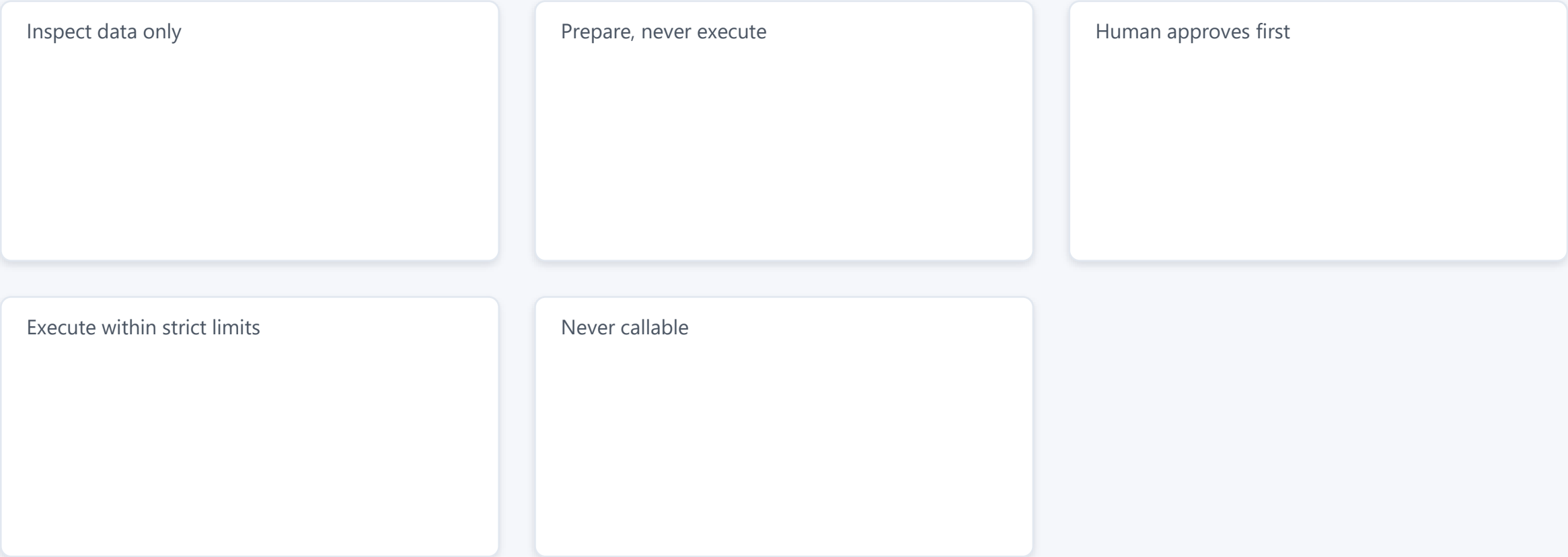
Infographic 2 — Agent Responsibility Card



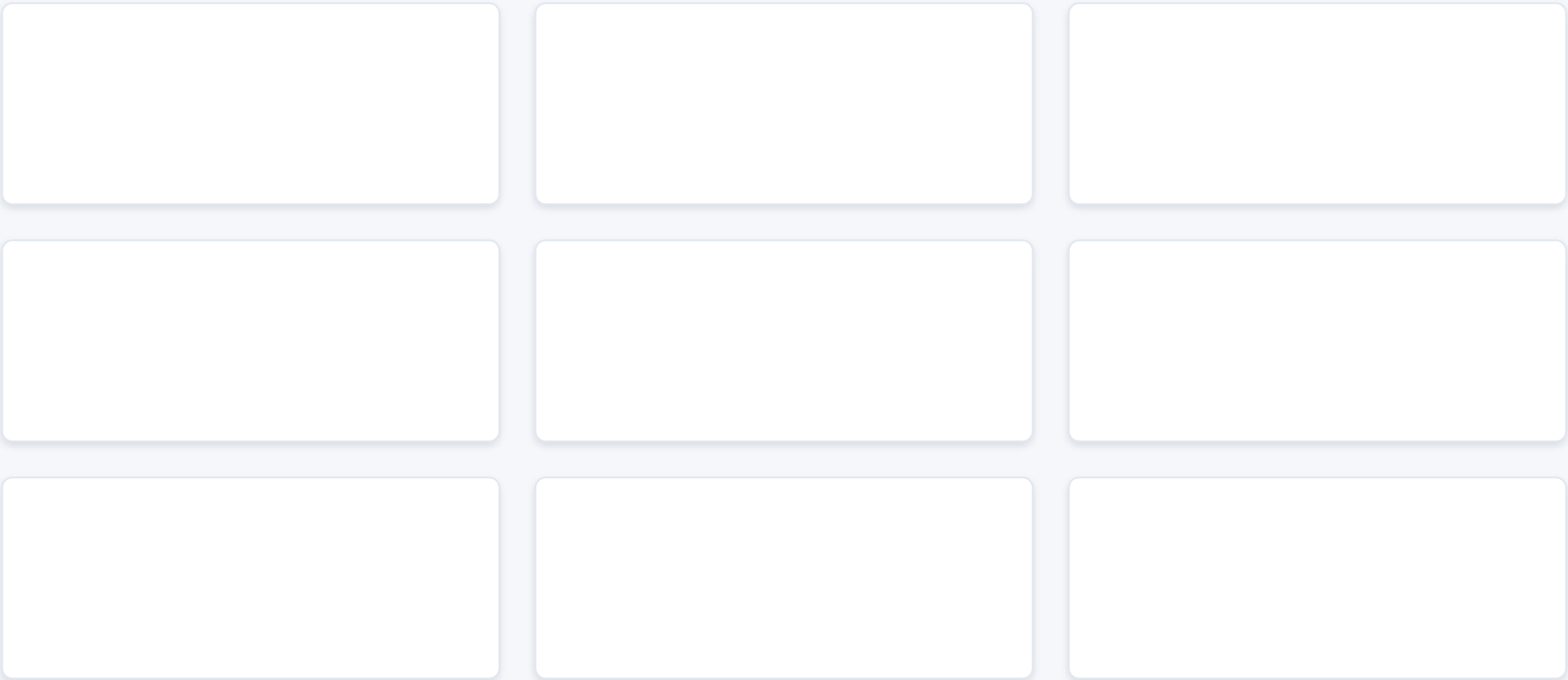
Infographic 3 — Pattern Selection Map



Infographic 4 — Tool Boundary Matrix



Infographic 5 — Enterprise Agent Control Plane



Infographic 6 — MCP / A2A / Framework Distinction

Framework = workflow control plane

MCP = tool / data / resource connector protocol

A2A = agent communication protocol

Governance = policy, identity, audit, approval

Infographic 7 — Failure-Safe Agent Workflow

REFERENCES

References — Frameworks & Protocols

References — Frameworks & Protocols

Framework and protocol positioning in this deck is based on official and primary documentation:

- LangGraph documentation positions LangGraph as an orchestration runtime focused on durable execution, streaming, human-in-the-loop, and persistence — docs.langchain.com/oss/python/langgraph/overview
- LangGraph persistence and interrupts documentation cover checkpointing, persistence, and pause/resume human-in-the-loop workflows — docs.langchain.com/oss/python/langgraph/persistence · docs.langchain.com/oss/python/langgraph/interrupts
- OpenAI Agents SDK documentation states that Agent plus Runner lets the SDK manage turns, tools, guardrails, handoffs, and sessions — openai.github.io/openai-agents-python/agents/
- OpenAI Agents SDK tracing documentation describes built-in tracing for LLM generations, tool calls, handoffs, guardrails, and custom events — openai.github.io/openai-agents-python/tracing/
- Microsoft Agent Framework overview describes it as combining AutoGen's agent abstractions with Semantic Kernel's enterprise features such as session state, type safety, filters, telemetry, and model support — learn.microsoft.com/en-us/agent-framework/overview/
- Google ADK documentation describes ADK as an open-source framework for building, debugging, and deploying reliable AI agents at enterprise scale — docs.cloud.google.com/gemini-enterprise-agent-platform/build/adk
- CrewAI documentation describes crews, flows, guardrails, memory, knowledge, and observability — docs.crewai.com
- MCP official documentation describes MCP as an open-source standard for connecting AI applications to external systems, including data sources, tools, and workflows — modelcontextprotocol.io/docs/getting-started/intro
- MCP specification describes server features including resources, prompts, and tools — modelcontextprotocol.io/specification/2025-11-25
- A2A official documentation describes Agent2Agent as an open standard for communication and collaboration between AI agents across frameworks and vendors — a2a-protocol.org/latest/
- A2A documentation comparing A2A and MCP states that MCP connects agents to tools/resources while A2A enables agent-to-agent collaboration — a2a-protocol.org/latest/topics/a2a-and-mcp/

References — Case Studies

All pattern case studies are drawn from primary or first-party sources:

- Harvey — harvey.ai: platform documentation, “25,000+ custom workflows”, customer stories (A&O Shearman, GSK Stockmann, Filip & Company)
- Klarna — openai.com/index/klarna and Klarna press release: “AI assistant handles two-thirds of customer service chats in its first month” (Feb 2024); 2025 rebalancing coverage
- LinkedIn Hiring Assistant — LinkedIn Engineering / InfoQ QCon talk: “Building LinkedIn’s First Production Agent”
- Replit Agent — langchain.com/breakoutagents/replit: multi-agent manager/editor/verifier architecture on LangGraph
- Uber Validator & AutoCover — Uber Developer Platform session, LangChain Interrupt 2025: 21,000 developer hours saved
- Anthropic Research — anthropic.com/engineering/multi-agent-research-system: lead agent + parallel subagents, 90.2% eval uplift, ~15× token usage (June 2025)
- OpenAI Agents SDK — openai.github.io/openai-agents-python (handoffs) and OpenAI’s “A practical guide to building agents”
- MetaGPT — arXiv 2308.00352 (ICLR 2024 oral); IBM Think: shared message pool with publish–subscribe
- Microsoft MAI-DxO — microsoft.ai/news/the-path-to-medical-superintelligence; arXiv 2506.22405 “Sequential Diagnosis with Language Models” (85.5% on 304 NEJM-CPC cases)
- GitHub Copilot coding agent — github.blog product announcement; docs.github.com: “Reviewing a pull request created by Copilot”